

Proceso de software y arquitectura

Proceso de software

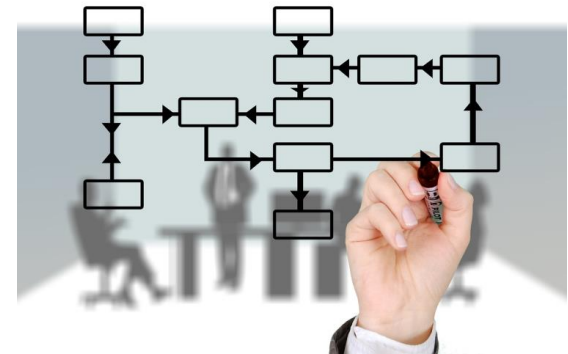
Factores importantes



Personas

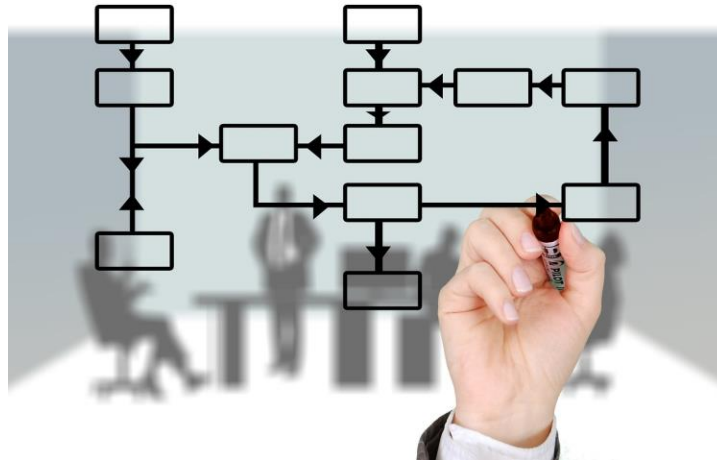


Herramientas



Procesos

Procesos



Ingeniería de software

Entender el problema
(Requerimientos)

Diseñar una solución

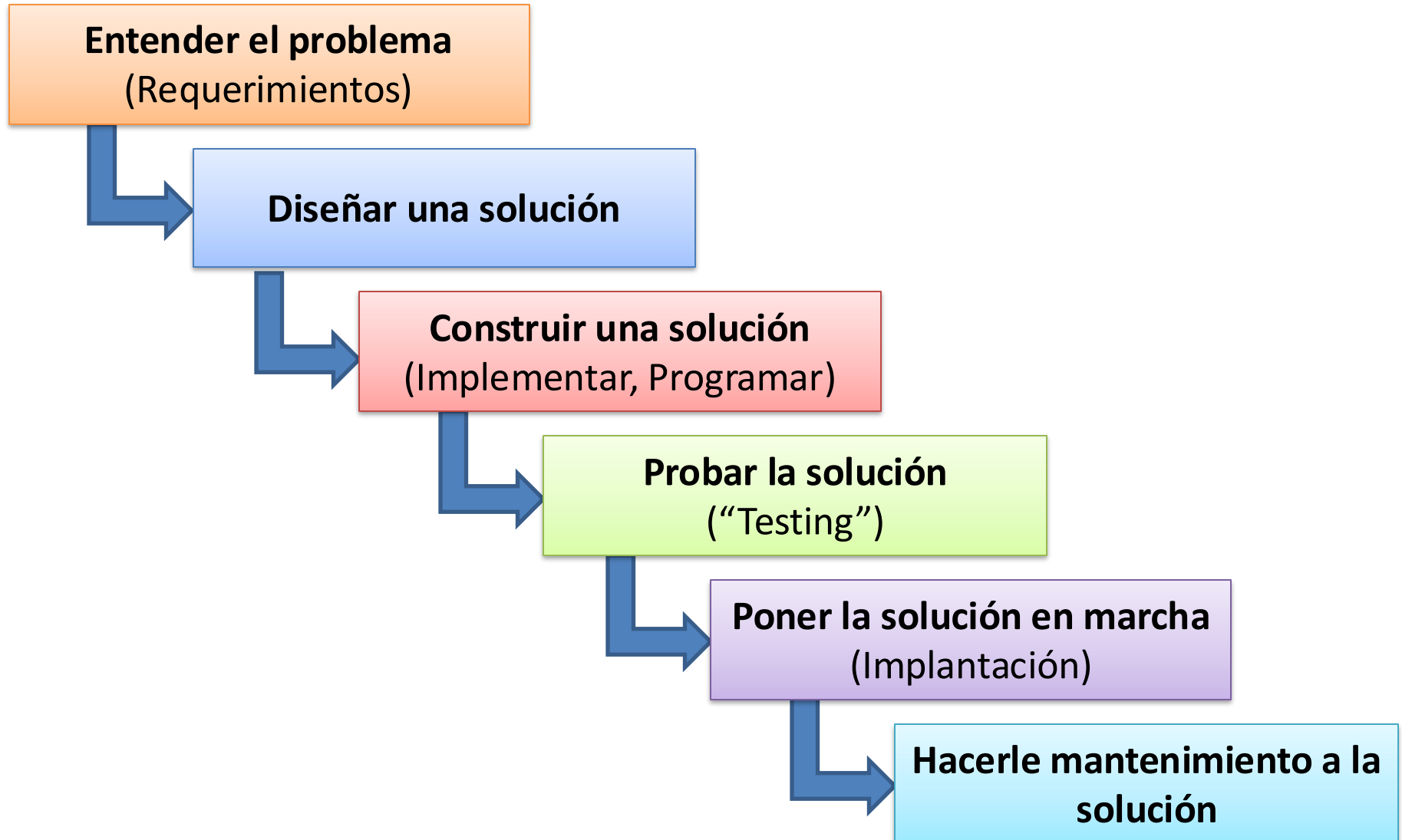
Construir una solución
(Implementar, Programar)

Probar la solución
("Testing")

**Poner la solución en
marcha**
(Implantación)

**Hacerle mantenimiento
a la solución**

Ingeniería de software tradicional



¿Cuál es problema?

- Desarrollo de software no es:
 - La construcción de un edificio
 - Un problema fácilmente predecible
- Desarrollo de software es:
 - Desarrollo de un **nuevo** producto

¿Por qué ocurre este problema?



¿Por qué ocurre este problema?

- Los clientes o usuarios no están seguros de lo que quieren
- Tienen dificultad para explicar todo lo que quieren y todo lo que saben
- Muchos detalles de lo que quieren sólo serán revelados durante el desarrollo
- Los detalles son extremadamente complejos para las personas
- A medida que ven el producto desarrollarse, cambian su opinión
- Fuerzas externas (tales como un producto o servicio de la competencia) impulsa a hacer cambios o mejoras

Clements, P., and Parnas, D. "A Rational Design Process: How and why to Fake It". IEEE Transactions on Soft. Eng. Feb 1986.

Desafíos de Ingeniería de Software

- Requerimientos cambiantes
- Rapidez de cambio
- Complejidad

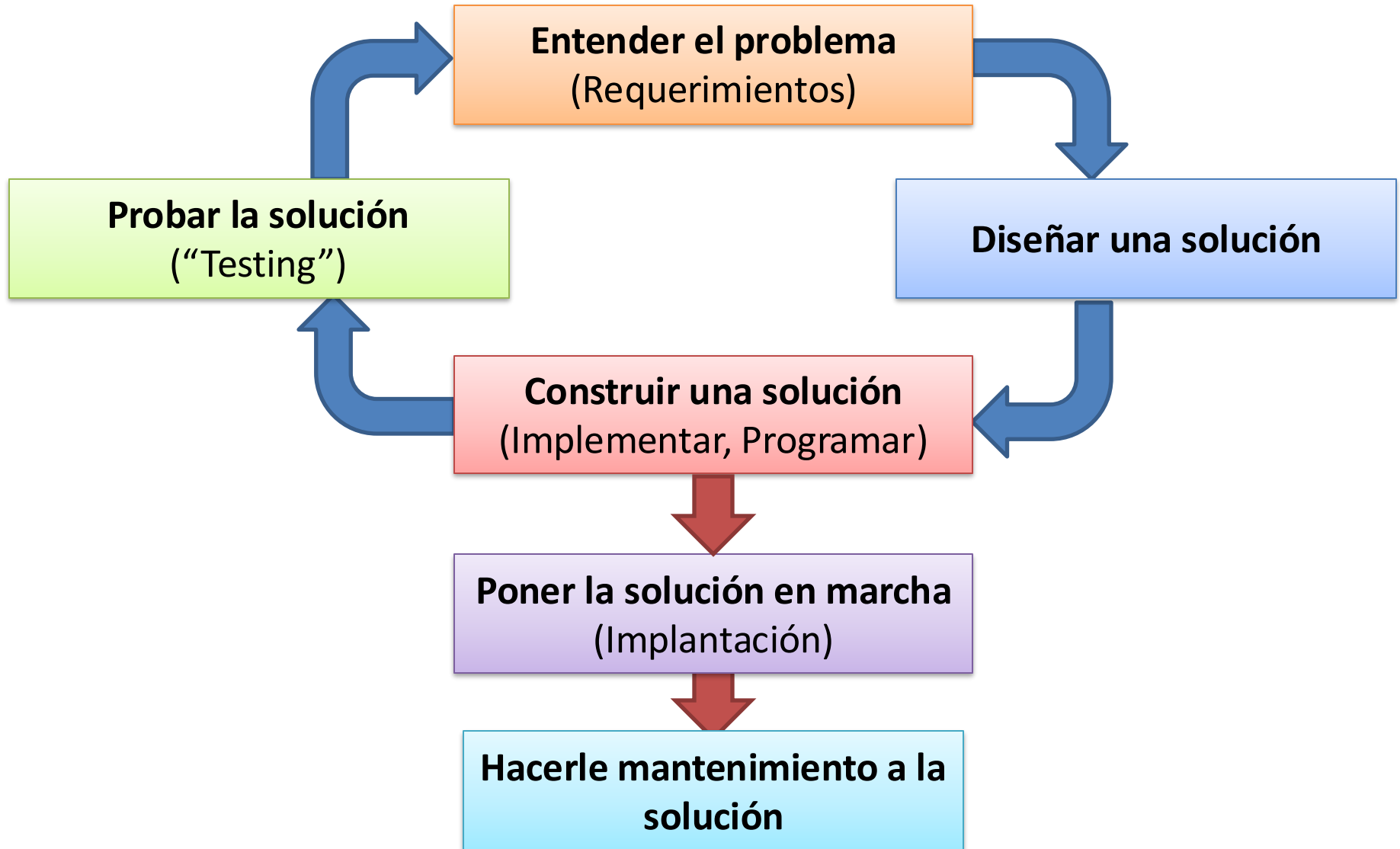


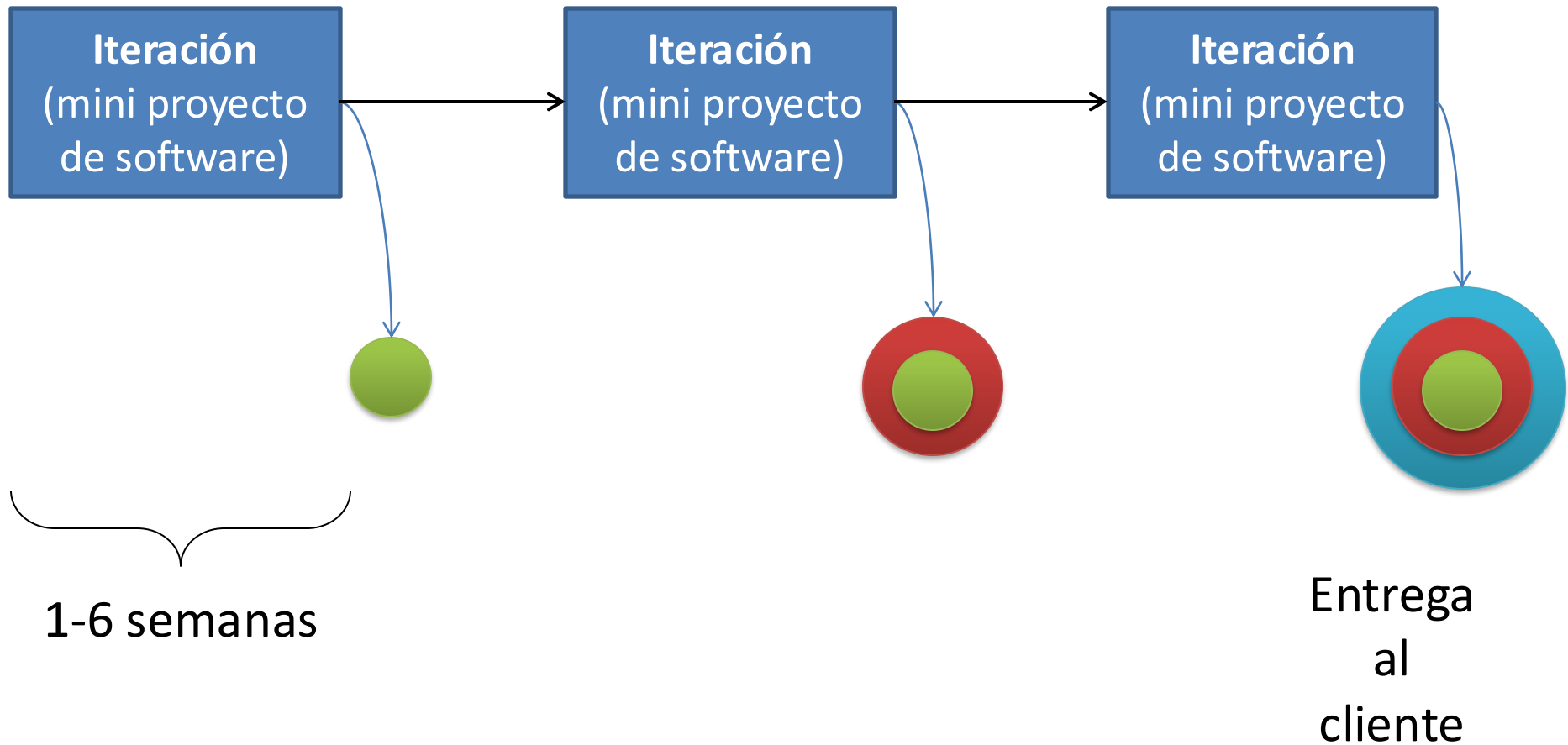
- Comunicación
- Disciplina
- Adaptabilidad
- Confianza



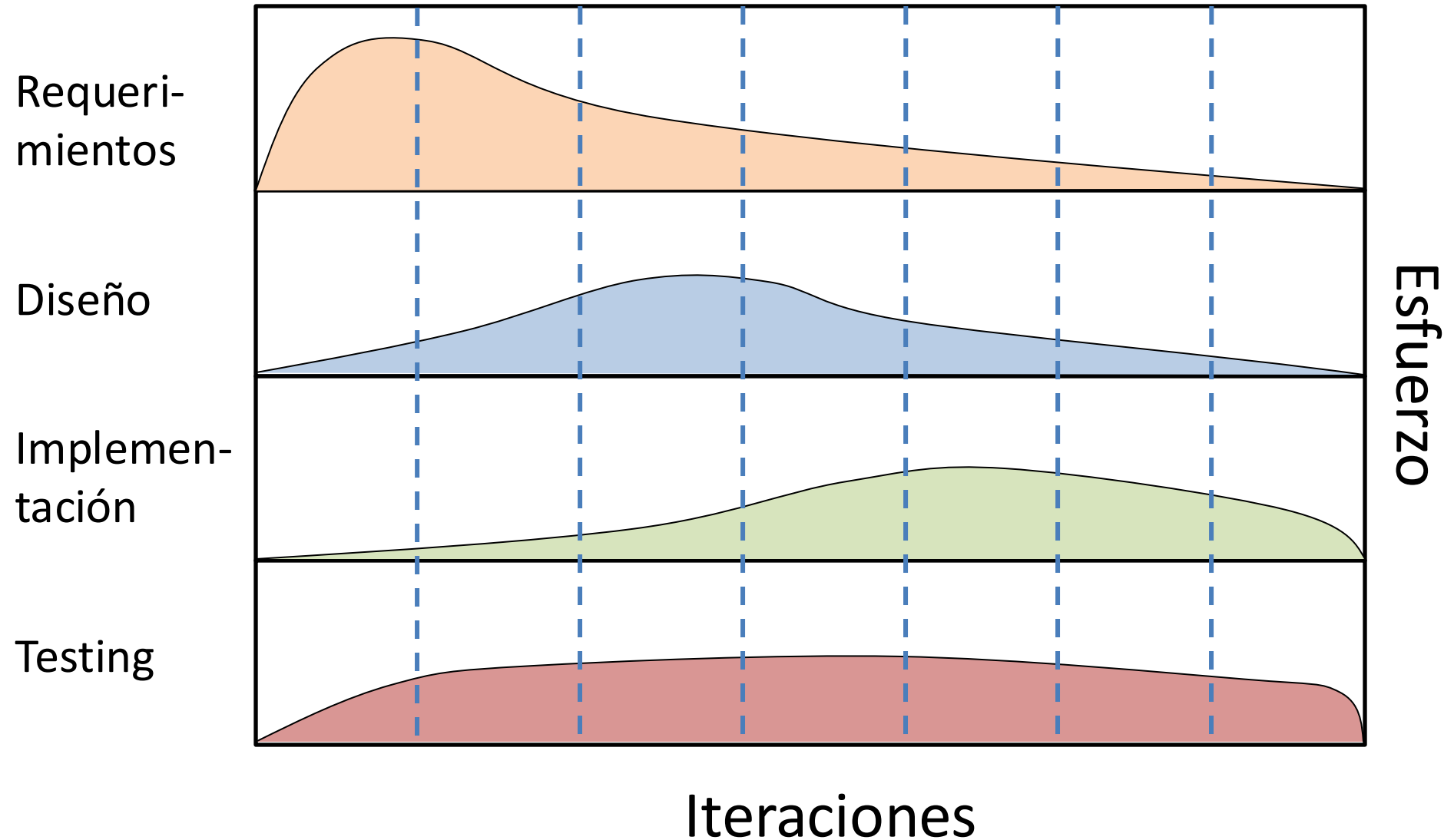
- Entender **bien** el problema
- Diseñar y desarrollar la solución **de calidad**
- Mantener la solución y **evitar su obsolescencia**

Ingeniería de software iterativa e incremental





Distribución de esfuerzo



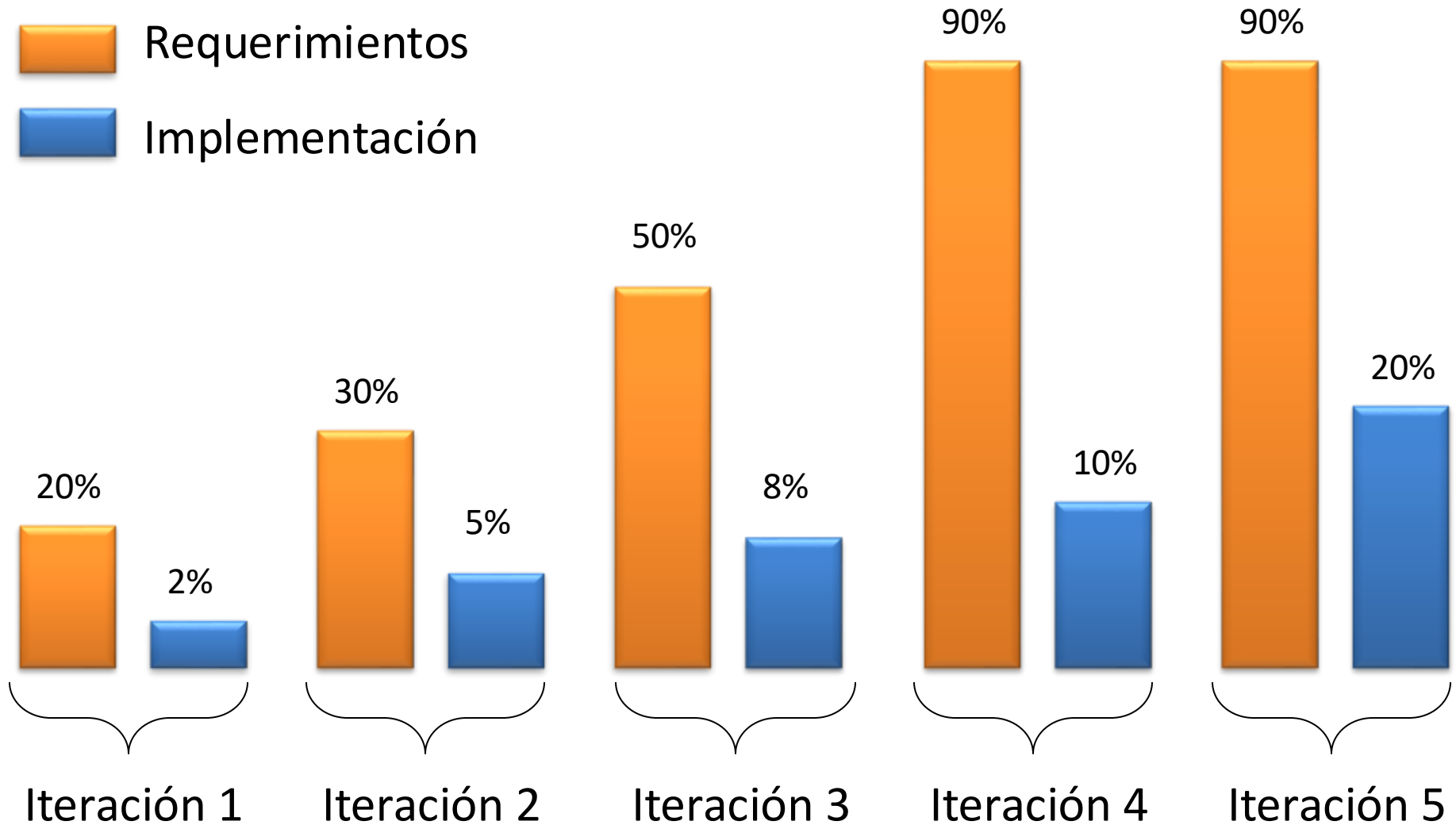
Distribución de progreso



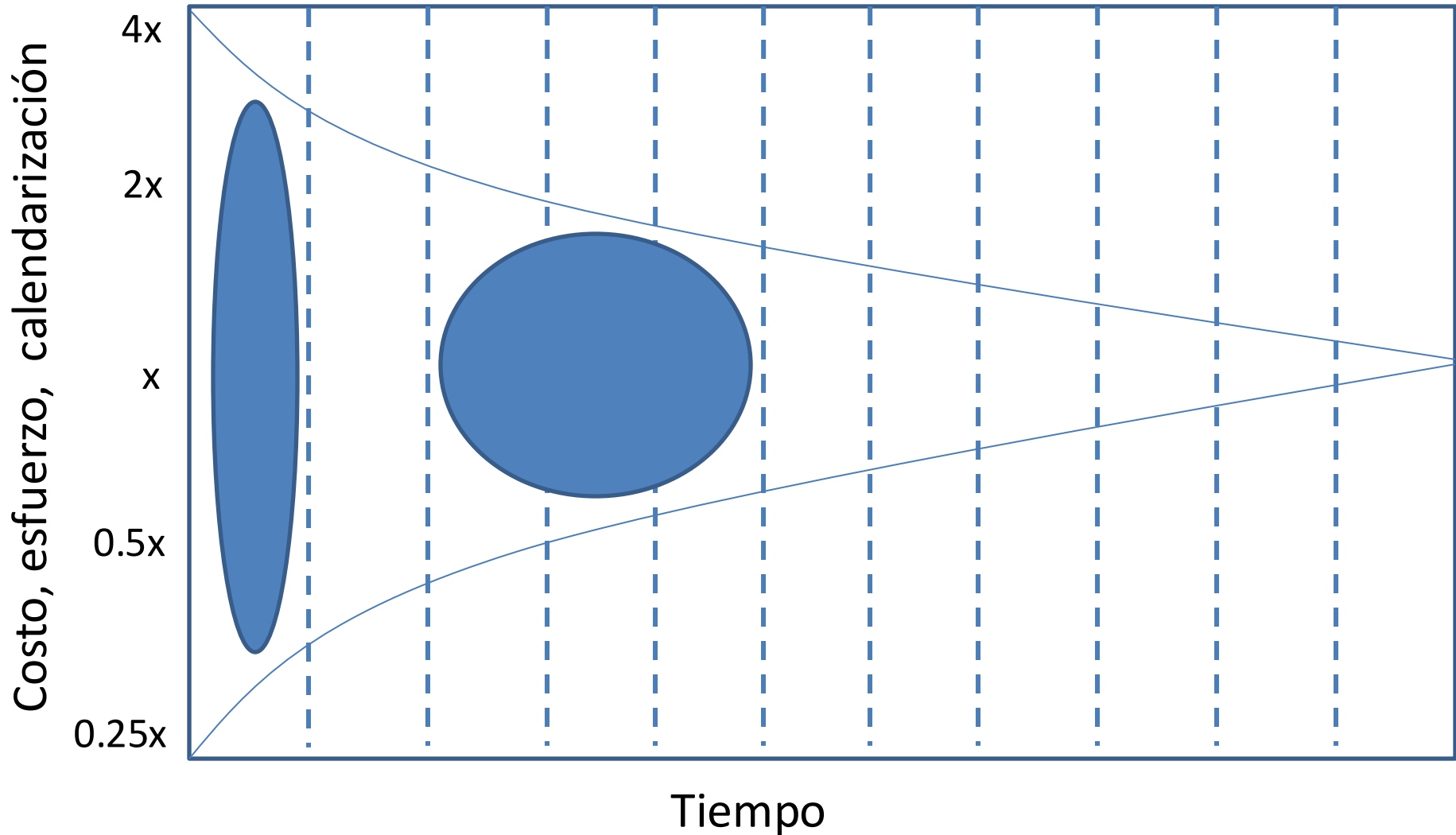
Requerimientos



Implementación

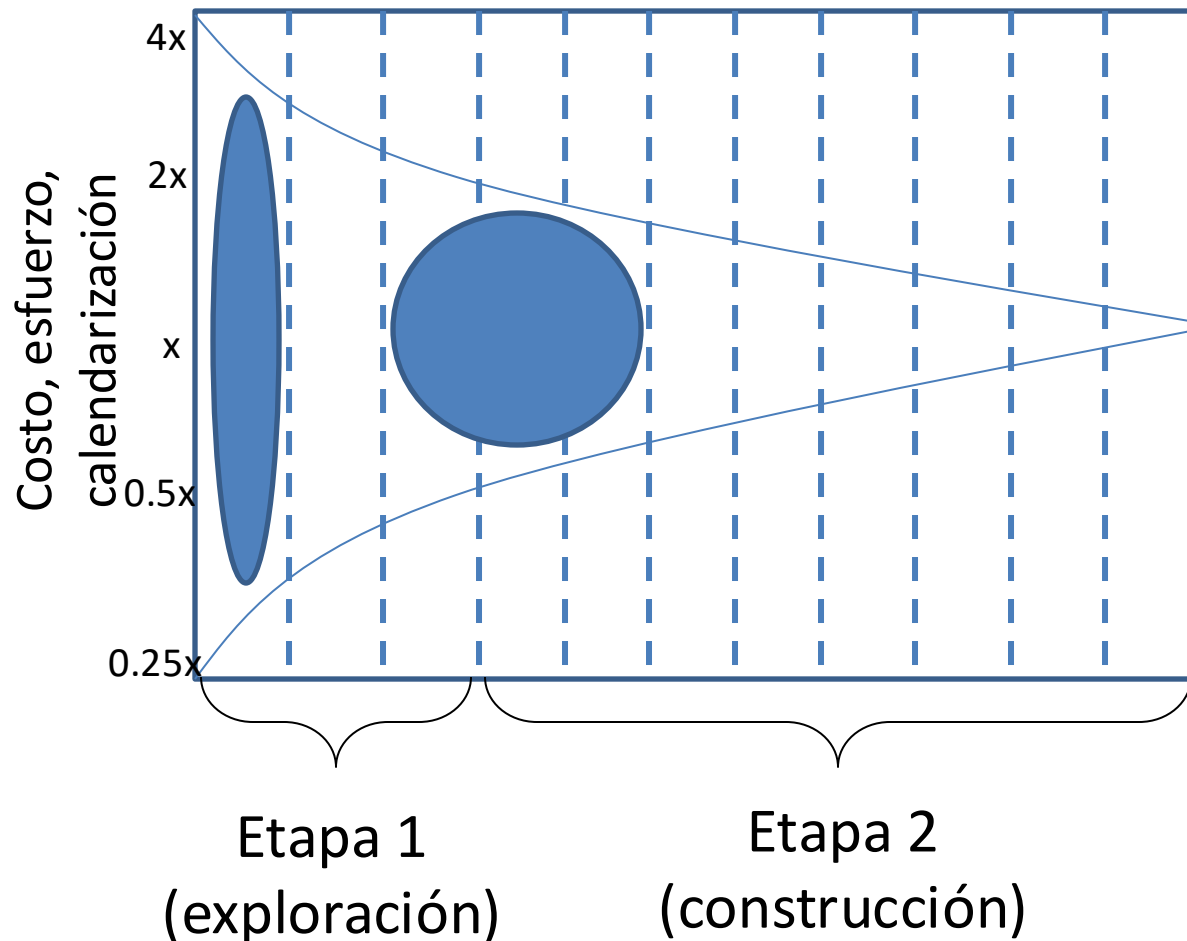


Cono de incertidumbre [McConnel 1998]



Contratos con costos fijos

- Ejecutar el proyecto en dos etapas



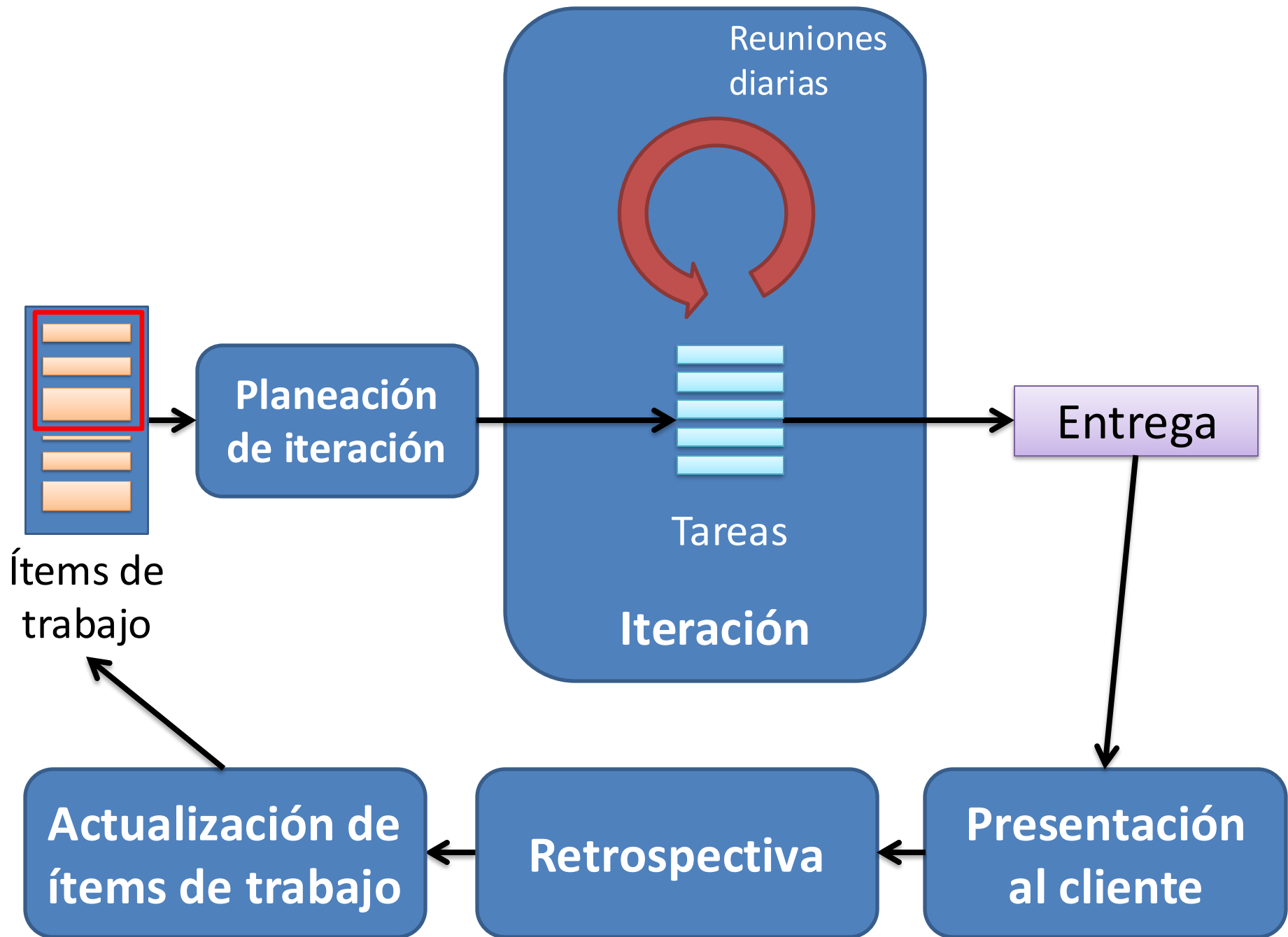
Métodos ágiles

- Iterativos e incrementales
- Tienden a ser más “livianos” y “veloces”
- Generación temprana y frecuente de valor
- **Factor humano**
- Algunos métodos ágiles
 - Scrum
 - Extreme Programming (XP)
 - DAD

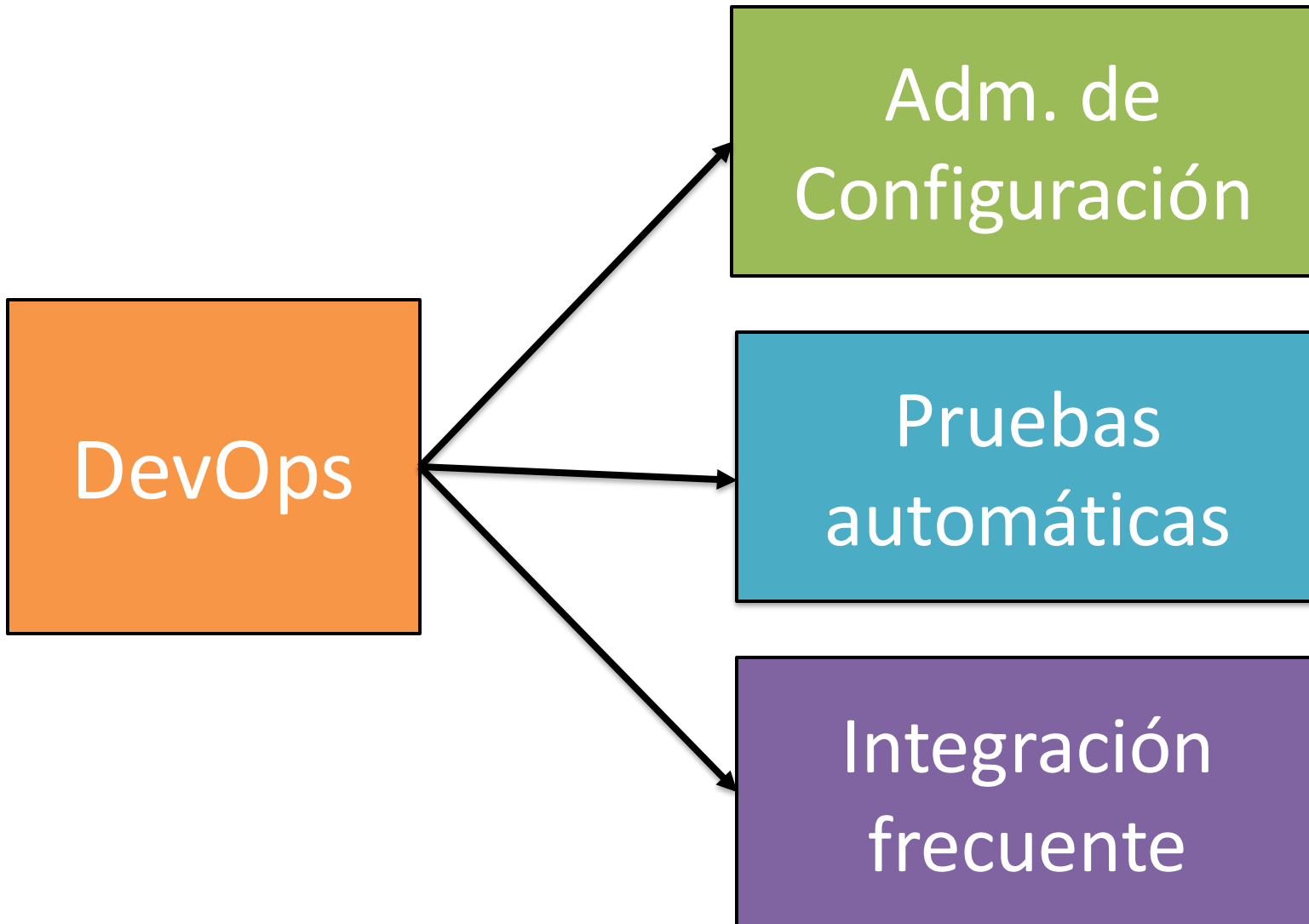
Propiedades esenciales

[Cockburn 2004]

1. Entregas frecuentes
2. Mejoramiento reflexivo
3. Comunicación
4. Seguridad personal
5. Concentración
6. Fácil acceso a usuarios expertos
7. DevOps



DevOps

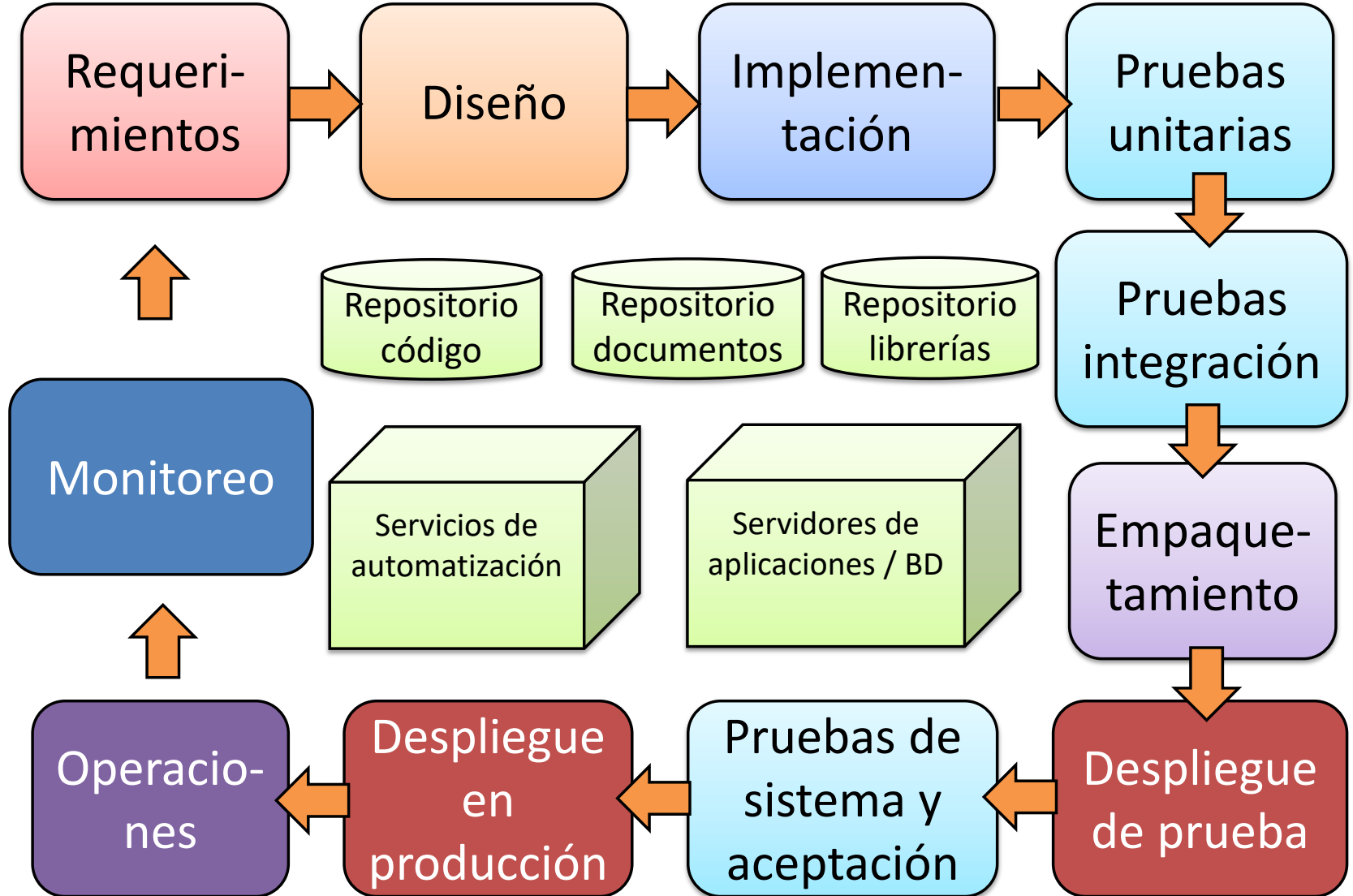


Herramientas



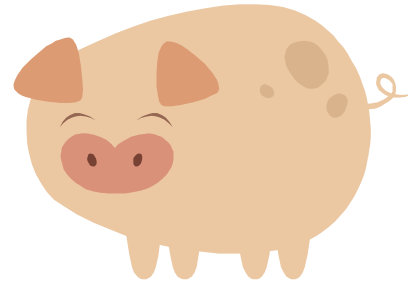
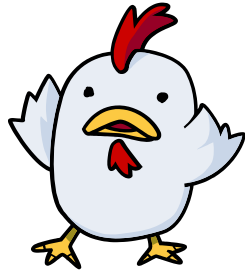
<https://www.amazon.com/Wenger-16999-Swiss-Knife-Giant/dp/B001DZTJRQ>

DevOps



¿Cómo conectar lo anterior con
arquitectura?





Una gallina y un cerdo discutían el nombre que le iban a dar a su nuevo restaurante . La gallina sugirió:
Restaurante Huevos con Jamón.



“No, gracias”, dijo el cerdo. “Con ese nombre, yo estaría absolutamente comprometido con el proyecto, mientras que tú estarías apenas involucrada”.

Algunos Roles



Resto de los
clientes y otras
personas de la
organización



Dueño del
producto



Administrador /
Líder de proyecto



Ingeniero de
requisitos



Arquitecto de
software

Equipo de desarrollo



Programador



Tester



Diseñador
Base de
Datos



Diseñador
UI/UX

Otros



Ingeniero de
procesos



Escritor
técnico



Diseñador
gráfico

Arquitecto vs Administrador

- Áreas en que el Arquitecto apoya al Administrador

Project Management Area (PMBOK)	Descripción
Integration	Coordinación entre los elementos del proyecto
Scope	Qué se va a hacer y qué no
Time	Asegurar que finalice a tiempo
Cost	Asegurar que se ejecute consistentemente al presupuesto
Quality	Asegurar que satisfaga todas las necesidades especificadas
Human Resources	Optimizar el uso de las personas del proyecto
Communications	Asegurar que la información fluya adecuadamente
Risk	Gestionar todo lo que puede salir mal
Procurement	Adquirir bienes y servicios desde fuera de la organización



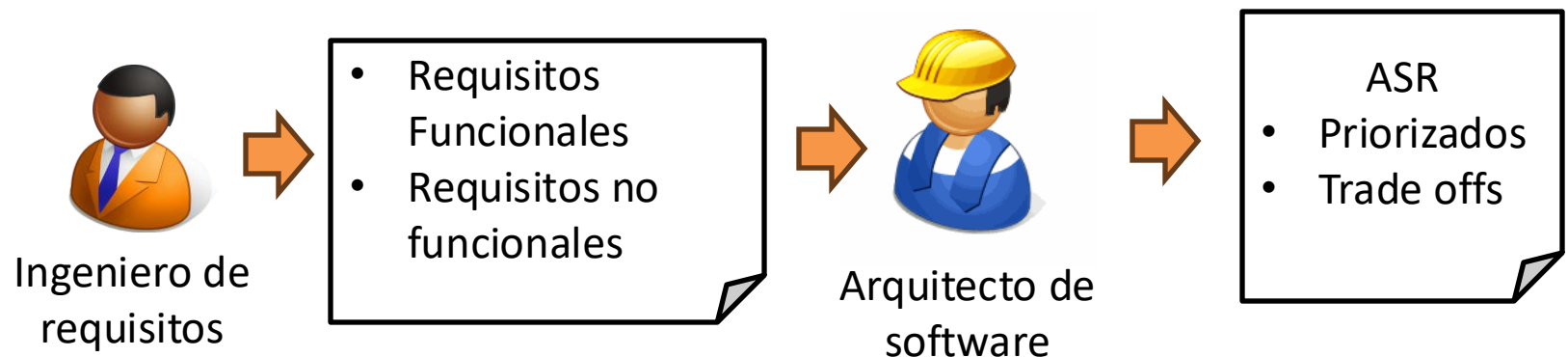
Administrador /
Líder de proyecto



Arquitecto de
software

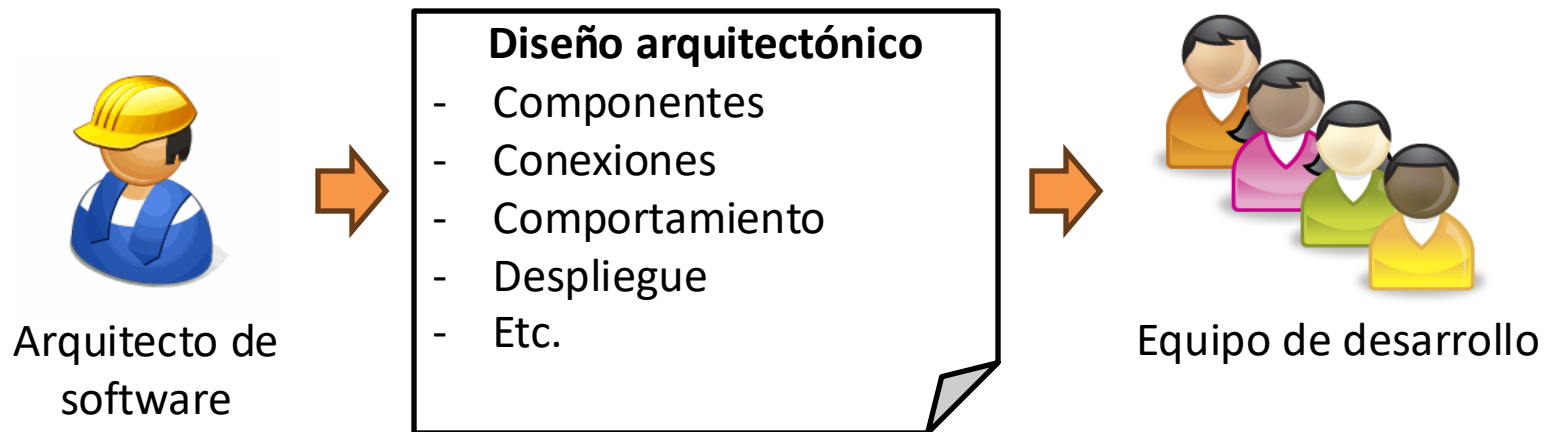
Arquitecto vs Ing. de requisitos

- Determinar los requisitos arquitectónicamente significativos (ASR)
 - Priorización
 - Análisis de trade-offs



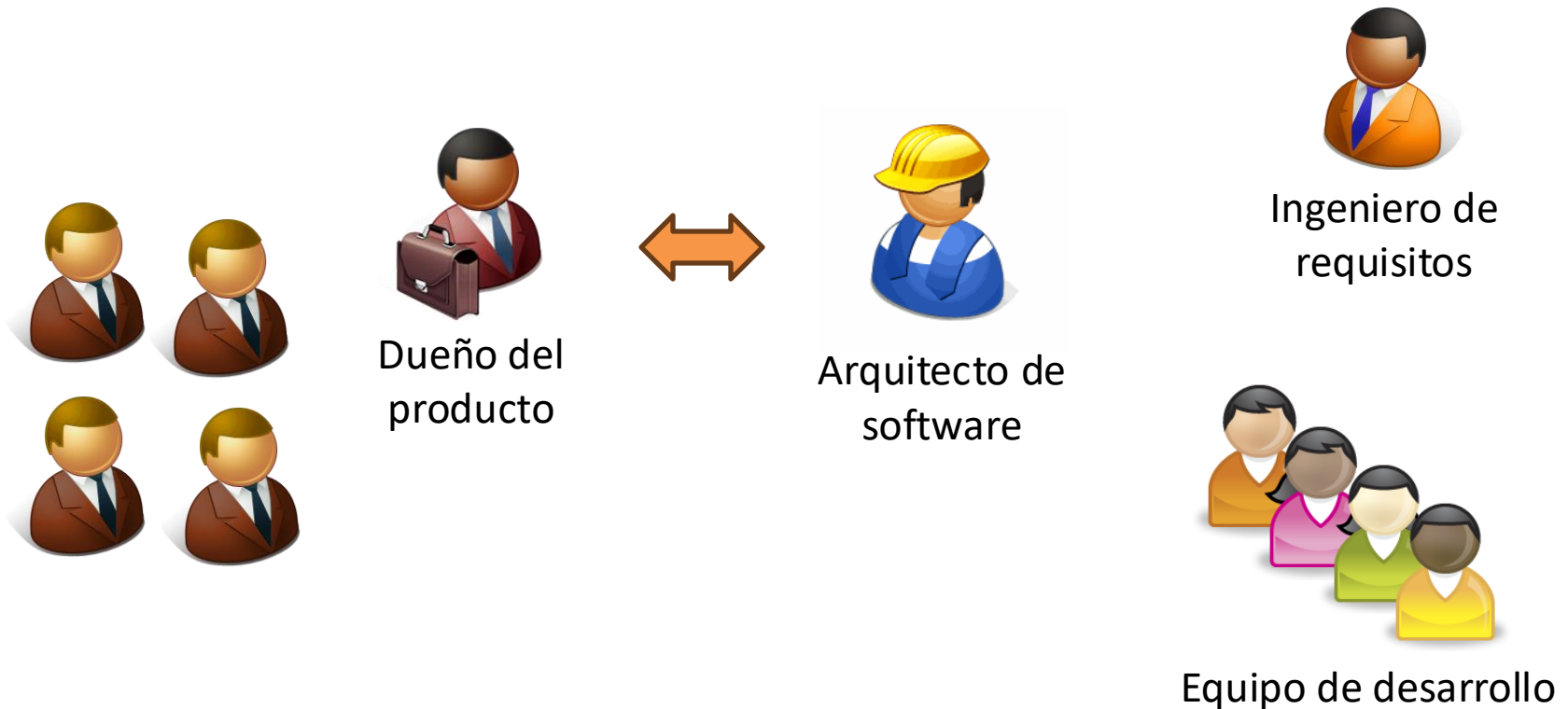
Arquitecto vs Equipo de desarrollo

- Lidera el diseño arquitectónico del sistema
- El equipo de desarrollo debe adherirse a lo que indica el Arquitecto



Arquitecto vs stakeholders

- Validación de arquitectura



Arquitecto vs otros roles



Arquitecto de
software



Otros



Ingeniero de
procesos



Escritor
técnico



Diseñador
gráfico

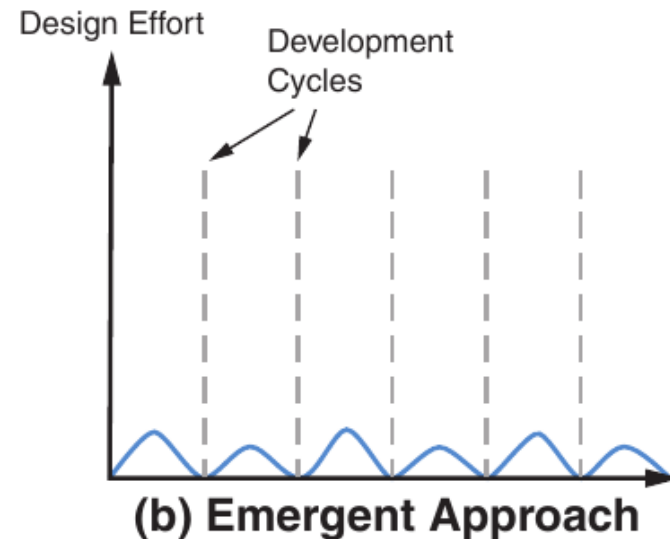
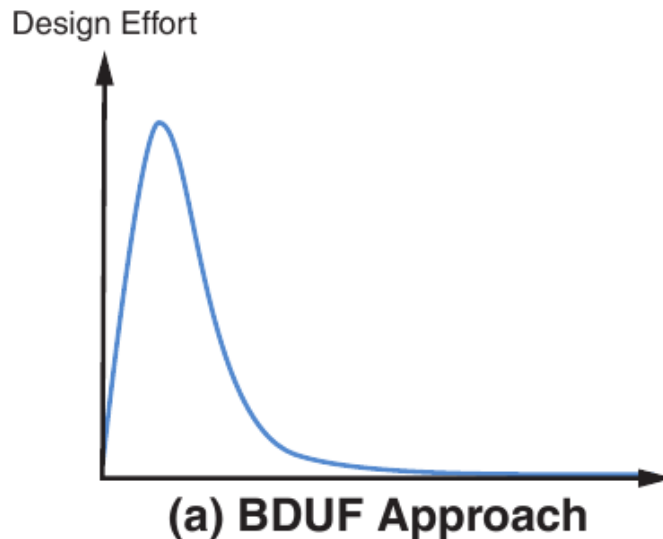
Arquitectura y Ciclo de vida

Modelo en Cascada y similares

- “Big Design Up Front” (BDUF)
- La arquitectura se establece al inicio del proyecto y no cambia

Métodos iterativos y ágiles

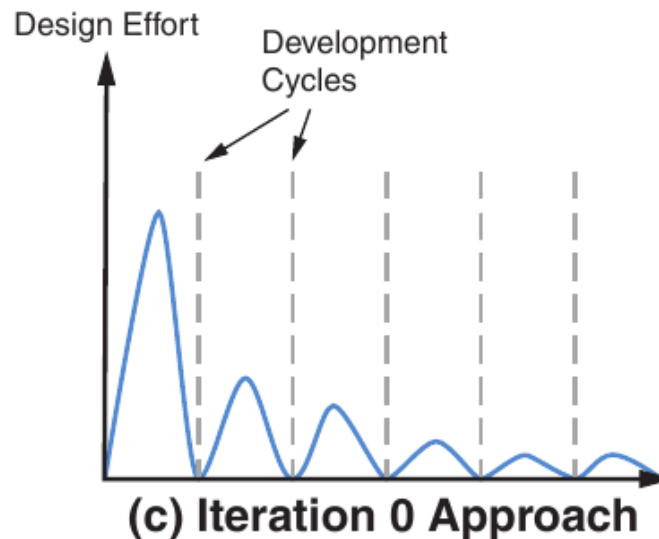
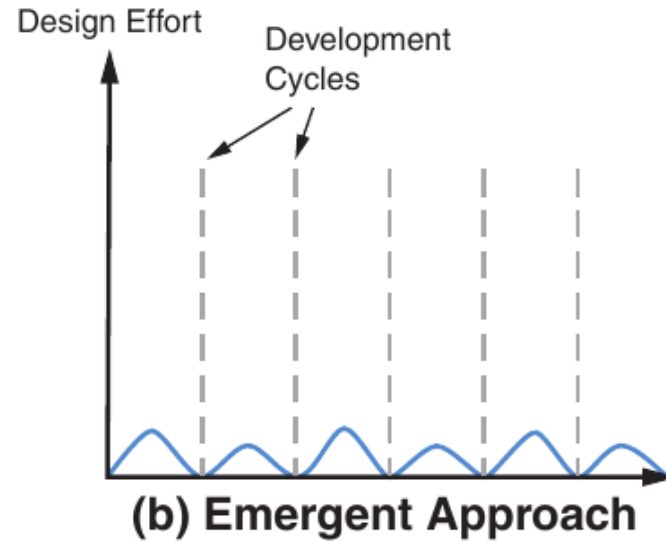
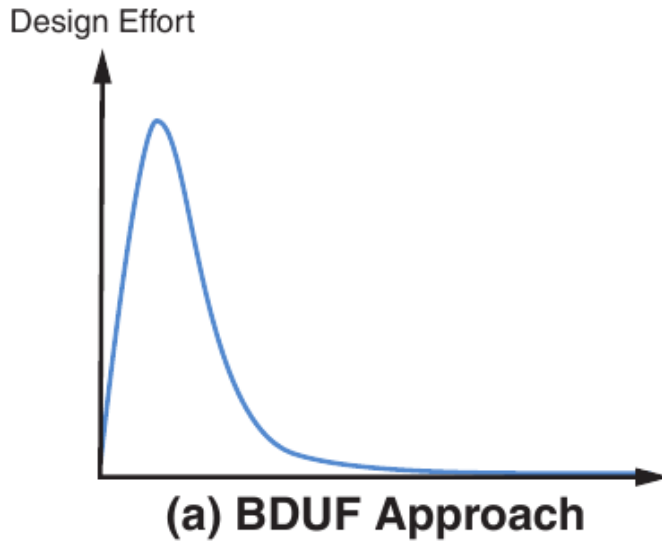
- Enfoque “emergente”
- Arquitectura creada por incrementos
- Versión final de la arquitectura al final del proyecto



¿Cuál opción es mejor?



Alternativa: Iteración 0



Arquitectura vs Métodos Ágiles

Principios Ágiles	Enfoque centrado arquitectura
Centrado en el cliente	✓
Adaptarse a requisitos cambiantes	✓
Entregas frecuentes	✓ (siempre y cuando se apoye en DevOps o automatización en general)
Trabajo conjunto con el cliente	✓
Priorizar interacción cara a cara sobre documentación	✗ (la documentación es esencial para proyectos complejos)
Software funcionando como medida principal de progreso	✓ (siempre y cuando no se dejen de lado los demás aspectos importantes: requisitos, diseño, pruebas, etc.)
Paso sostenible	✓
Equipos auto-organizados como fuente principal de un buen diseño	✗ (para sistemas complejos se requieren personas capacitadas y una organización más formal)
Mejoramiento continuo	✓

Resumiendo

- Ingeniería de software
 - Personas + Procesos + Herramientas
- Métodos tradicionales + iterativos ágiles
 - Nivel de burocracia
 - Cómo atacar incertidumbre
- DevOps
 - Automatizar todo lo que se pueda
- Rol del Arquitecto de software
- Arquitectura vs agilismo
 - Iteración 0: enfoque “balanceado”

Ejercicio – Use el conocimiento que tenga actualmente

- Para las propuestas de sistemas de software de cada equipo
 - Definir y priorizar los atributos de calidad (alta, media, baja)
 - Definir los principales componentes y conexiones
 - ¿Cómo la arquitectura propuesta satisface cada atributo de calidad?